# Тема. Применение непрерывного вейвлет-преобразования для поиска краев объектов на изображении

## Прудникова Анастасия 214

Вариант 23

### §1 Схемы подразделений

Пусть $s \in \mathbf{N}$ и $\mathbf{Z}^s$ -- целочисленная решетка. Схема подразделений определяется заданной последовательностью $a = \{a_\alpha\}_{\alpha \in \mathbf{Z}^s}$. Мы будем предполагать, что $\operatorname{supp}(a) = \{\alpha : a_\alpha \neq 0\}$ -- конечное множество. Обозначим $l_\infty(\mathbf{Z}^s)$ линейное нормированное пространство ограниченных последовательностей $\lambda = \{\lambda_\alpha\}_{\alpha \in \mathbf{Z}^s}$, в котором норма определяется равенством

$$\|\lambda\|_\infty = \sup_{\alpha \in \mathbf{Z}^s} |\lambda_\alpha|.$$

Введем в рассмотрение оператор $\mathrm{S} \ : \ l_\infty(\mathbf{Z}^s) \to l_\infty(\mathbf{Z}^s)$, который определим формулой [1]

$$(\mathrm{S}\lambda)_\alpha = \sum_{\beta \in \mathbf{Z}^s} a_{\alpha - 2\beta} \lambda_\beta, \ \lambda \in l_\infty(\mathbf{Z}^s).$$

**Определение 1.** Будем говорить, что схема подразделений [1]

$$\lambda^m = \mathrm{S}\lambda^{m-1}, \ m = 1, 2, \ldots, \ \lambda^0 = \lambda$$

сходится для $\lambda \in l_\infty(\mathbf{Z}^s)$, если существует непрерывная функция $f_\lambda$, определенная на $\mathbf{R}^s$ такая, что

$$\lim_{m \to +\infty} \left\| f_\lambda\left(\frac{\cdot}{2^m}\right) - \lambda^m \right\|_\infty = 0.$$

Мы будем последовательность $\{a_\alpha\}_{\alpha \in \mathbf{Z}^s}$ называть **маской схемы подразделений**, а $\mathrm{S}$ -- оператором схемы подразделений [1].

Введем в рассмотрение последовательность

$$(\uparrow \lambda)_\alpha = \begin{cases} \lambda_\beta, \ \text{если } \alpha = 2\beta \\ 0, \ \text{в противном случае.} \end{cases}$$

Тогда

$$(S\lambda)_\alpha = \sum_{\beta \in \mathbf{Z}^s} a_{\alpha - 2\beta} \lambda_\beta = \sum_{\beta \in \mathbf{Z}^s} a_{\alpha - \beta} (\uparrow \lambda)_\beta = (a * (\uparrow \lambda))_\alpha.$$

Из определения 1 следует, что $f_\lambda(2^{-m}\alpha) \approx \lambda_\alpha^m$, где $\alpha \in \mathbf{Z}^s$.

### §2. Поиск краев объектов на изображении

**Определение 2.** Под вейвлетом мы будем понимать функцию $\psi \in \mathrm{L}_1(\mathbf{R}^2) \cap \mathrm{L}_2(\mathbf{R}^2)$, удовлетворяющую условию $\int\limits_{\mathbf{R}^2} \psi(\mathrm{x}) d\mathrm{x} = 0$.

**Определение 3.** Для функции двух переменных $f(x, y)$ непрерывное вейвлет-преобразование этой функции определим равенством [2]

$$W_s f(x, y) = \iint\limits_{\mathbf{R}^2} f(x - u, y - v) \psi_s(u, v) du dv, \ \ i = 1, 2,$$

где $\psi_s(u, v) = \frac{1}{s^2} \psi\left(\frac{u}{s}, \frac{v}{s}\right)$, $i = 1, 2$.

Мы будем рассматривать специальный класс вейвлет-функций [2]

$$\psi^1(u, v) = \psi(\sqrt{u^2 + v^2}) \cos\theta, \ \ \psi^2(u, v) = \psi(\sqrt{u^2 + v^2}) \sin\theta,$$

где $\theta$ -- полярный угол точки $(u, v)$, а носитель функции $\psi \in \mathrm{L}_1(\mathbf{R}) \cap \mathrm{L}_2(\mathbf{R})$ будем предполагать компактным, содержащемся в $[0; +\infty)$. В этом случае

$$\iint\limits_{\mathbf{R}^2} \psi^i(u, v) du dv = \int\limits_0^{+\infty} r\psi(r) dr \int\limits_0^{2\pi} k_i(\theta) d\theta = 0,$$

где $k_1(\theta) = \cos\theta$, $k_2(\theta) = \sin\theta$. Таким образом, обе функции являются вейвлетами.

Пусть

$$W_s^i f(x,y) = \iint\limits_{\mathbf{R}^2} f(x-u, y-v)\psi_s^i(u,v)dudv, \;\; i = 1,2.$$

и

$$M_s f(x,y) = \sqrt{|W_s^1 f(x,y)|^2 + |W_s^2 f(x,y)|^2}.$$

### §3. Вычисление $W_s f$

Заметим, что

$$W_s f(n,m) = \iint\limits_{\mathbf{R}^2} f(u,v)\psi_s(n-u, m-v)dudv = \sum_{k,l} f(k,l) \iint\limits_{[k;k+1]\times[l,l+1]} \psi_s(n-u, m-v)dudv =$$

$$= \sum_{k,l} f(k,l) \iint\limits_{[n-k-1;n-k]\times[m-l-1,m-l]} \psi_s(u,v)dudv = \sum_{k,l} f(n-k, m-l)\psi_{k,l}^s,$$

где

$$\psi_{k,l}^s = \iint\limits_{[k-1;k]\times[l-1,l]} \psi_s(u,v)dudv \approx \frac{1}{r^2}\sum_{i_1=0}^{r-1}\sum_{i_2=0}^{r-1}\psi_s\left(k-1+\frac{2i_1+1}{2r}, l-1+\frac{2i_2+1}{2r}\right), \;\; r \in \mathbf{N}.$$

### §4. Алгоритм определения края

**Дано:** Изображение $f$ и порог $R > 1$, близкий к 1. Значения $s_1 > s_2 > \dots > s_J$. Порог $T$.

1. Для $j = 1, 2, \dots, J$ находим $M_{s_j} f$.
2. Находим точки, для которых $M_{s_j} f(x,y) \geqslant T$, $j = 1, 2, \dots, J$.
3. Среди найденных на предыдущем шаге точек оставляем те (точки края), которые удовлетворяют условию

$$\frac{1}{R} \leqslant \frac{M_{s_j} f(x,y)}{M_{s_l} f(x,y)} \leqslant R, \;\; j,l = 1,2,\dots,J.$$

## Задание и варианты

**Задание 1**. В точках $2^{-m}k, \; k \in \mathbf{Z}, \;\; m \in \mathbf{N}$ найти значения предельной функции $\psi$ схемы подразделений, в которой дана маска схемы подразделений $a = \{a_\alpha\}_{\alpha\in\mathbf{Z}}$ и последовательность $\lambda^0 = \delta_{0,t}$, $t > 0$, выбрана так, чтобы $\mathrm{supp}\psi \subset (0; +\infty)$. Построить график предельной функции и, используя кусочно-линейную интерполяцию, написать функцию на языке Python, вычислющую значения предельной функции в произвольной точке $x \in \mathbf{R}$.

**Задание 2**. Найти значения

$$\psi_{k,l}^{s,i} = \iint\limits_{[k-1;k]\times[l-1,l]} \psi_s^i(u,v)dudv \approx \frac{1}{r^2}\sum_{i_1=0}^{r-1}\sum_{i_2=0}^{r-1}\psi_s^i\left(k-1+\frac{2i_1+1}{2r}, l-1+\frac{2i_2+1}{2r}\right), \;\; r \in \mathbf{N},$$

где $\psi^1(u,v) = \psi(\sqrt{u^2+v^2})\cos\theta$, $\psi^1(u,v) = \psi(\sqrt{u^2+v^2})\sin\theta$, для $s = 2, 4, 8$.

**Задание 3**. Используя фильтры $\psi_{k,l}^{s,i}$, найденные в задании 2, реализовать алгоритм поиска краев объектов на изображении. Протестировать алгоритм на заданном изображении, хранящемся в папке DetectEdge.

**Вариант № k**. $a_\alpha = \begin{cases} \frac{1}{6}+\omega, \; \alpha = 0; \\ \frac{5}{6}-\omega, \; \alpha = 1; \\ \frac{5}{6}-\omega, \; \alpha = 2; \\ \frac{1}{6}+\omega, \; \alpha = 3; \\ 0, \; \alpha \notin \{0,1,2,3\}, \end{cases}$

где

$$\omega = -\frac{1}{6} + \frac{2\cdot k + N}{4\cdot(N+1)}, \;\; k = 1,2,\dots,N,$$

а $N$ - количество человек в группе.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
import cmath
import math
import torch
from scipy import signal
from PIL import Image
%matplotlib inline
```

```
def Mask(omega):
    b = np.zeros(4)
    b[0] = 1./ 6. + omega
    b[1] = 5./ 6. - omega
    b[2] = 5./ 6. - omega
    b[3] = 1./ 6. + omega
    a = pd.Series(b, index = (0, 1, 2, 3))
    return a
```

```
def psi(x, Lambda, X):
    ''' Линейная интерполяция '''
    y = 0
    Y = np.nonzero(X - x > 0)
    if (len(Y[0]) > 1 and len(Y[0]) < len(X) - 1):
        i0 = Y[0][0] - 1
        i1 = Y[0][0]
        y0 = Lambda[i0]
        x0 = X[i0]
        y1 = Lambda[i1]
        x1 = X[i1]
        y = y0 + (y1 - y0) * (x - x0) / (x1 - x0)
    return y

def psi_1(u, v, Lambda, X):
    c = complex(u, v)
    polar_coordinates = cmath.polar(c)
    y = psi(polar_coordinates[0], Lambda, X) * np.cos(polar_coordinates[1])
    return y

def psi_s_1(u, v, Lambda, X, s):
    y = (1 / (s ** 2)) * psi_1(u / s, v / s, Lambda, X)
    return y

def psi_2(u, v, Lambda, X):
    c = complex(u, v)
    polar_coordinates = cmath.polar(c)
    y = psi(polar_coordinates[0], Lambda, X) * np.sin(polar_coordinates[1])
    return y

def psi_s_2(u, v, Lambda, X, s):
    y = (1 / (s ** 2)) * psi_2(u / s, v / s, Lambda, X)
    return y
```

```
def psi_interp(x, Lambda, X):
    y = 0
    Y = np.nonzero(X - x > 0)
    if (len(Y[0]) > 1 and len(Y[0]) < len(X) - 1):
        i0 = Y[0][0] - 1
        i1 = Y[0][0]
        y0 = Lambda[i0]
        x0 = X[i0]
        y1 = Lambda[i1]
        x1 = X[i1]
        y = y0 + (y1 - y0) * (x - x0) / (x1 - x0)
    return y
```

## Построение функции с компактным носителем, содержащемся в $(0; +\infty)$

Рассмотрим схему подразделений

$$\lambda^j_\alpha = \sum_\beta a^{j-1}_{\alpha-2\beta} = \sum_\beta a_{\alpha-\beta}(\uparrow \lambda^{j-1})_\beta,$$

что можно записать в виде

$$\lambda^j = a * (\uparrow \lambda^{j-1}).$$

Пусть $\operatorname{supp} a \subset [-r; r] \cap \mathbf{Z}$, $\operatorname{supp} \lambda^0 \subset [k^0_0; k^0_1] \cap \mathbf{Z}$. Ясно, что ненулевые значения $\lambda^j_\alpha$ могут быть полученны лишь при $\alpha \in [k^j_0; k^j_1]$, где $k^j_0 = 2k^{j-1}_0 - r$, $k^j_1 = 2k^{j-1}_1 + r$. Таким образом,

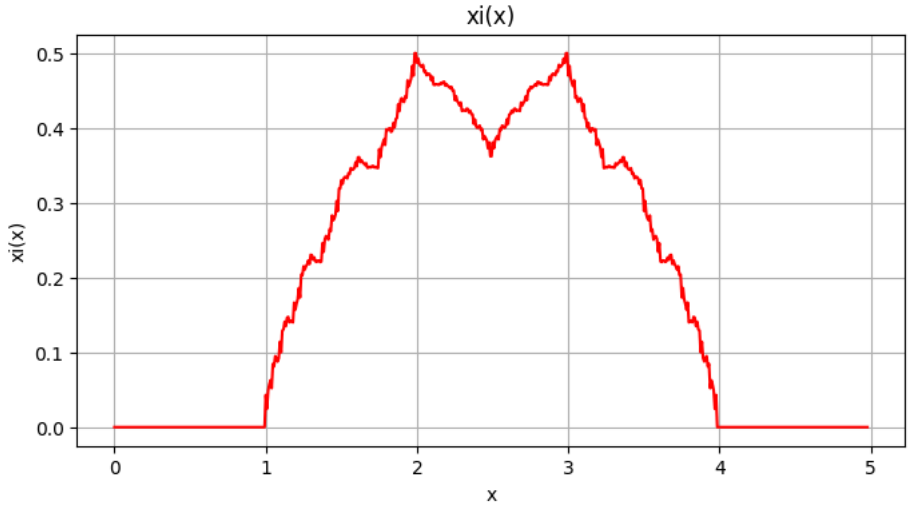
$$k^j_0 = 2^j k^0_0 - r(2^j - 1)$$

$$k_1^j = 2^j k_1^0 + r(2^j - 1).$$

```
def Subdivision(Lambda0, mask, m):
    Lambda = Lambda0.copy()
    for _ in range(m):
        L_new = []
        n = len(Lambda)
        for k in range(2 * n + len(mask) - 2):
            s = 0
            for j in mask.index:
                if (k - j) % 2 == 0:
                    i = (k - j) // 2
                    if 0 <= i < n:
                        s += mask[j] * Lambda[i]
            L_new.append(s)
        Lambda = np.array(L_new)
    return Lambda
```

```
k = 23
N = 28
omega = -1/6 + (2*k + N) / (4*(N + 1))
mask = Mask(omega)
print(f"omega = {omega:.6f}")

t = 1
m = 7
Lambda0 = np.zeros(3)
Lambda0[t] = 1
Lambda = Subdivision(Lambda0, mask, m)
X = np.linspace(0, len(Lambda) - 1, len(Lambda)) / (2 ** m)
```

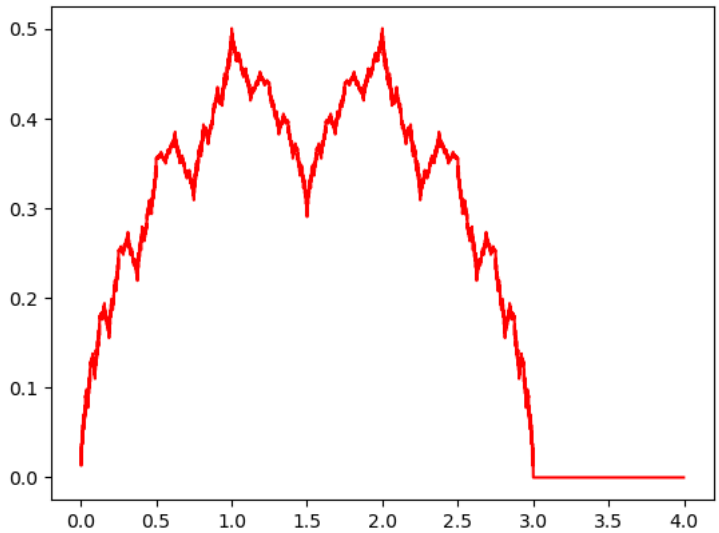
```
plt.figure(figsize=(8, 4))
plt.plot(X, Lambda, '-r')
plt.title('xi(x)')
plt.xlabel('x')
plt.ylabel('xi(x)')
plt.grid(True)
plt.show()

x_test = 0.75
y_test = psi(x_test, Lambda, X)
print(f"xi({x_test}) = {y_test}")
```




```
xi(0.75) = 0.0
```

```
plt.plot(X, Lambda, '-r')
```

```
[<matplotlib.lines.Line2D at 0x1748f1d7d90>]
```




```
x_max = X.max()
r = 10
h = 1.0 / r
```

```
M, r = 64, 8

filter_psi_s_1_2 = pd.DataFrame(np.zeros((M, M), dtype=float))
filter_psi_s_2_2 = pd.DataFrame(np.zeros((M, M), dtype=float))
filter_psi_s_1_4 = pd.DataFrame(np.zeros((M, M), dtype=float))
filter_psi_s_2_4 = pd.DataFrame(np.zeros((M, M), dtype=float))
filter_psi_s_1_8 = pd.DataFrame(np.zeros((M, M), dtype=float))
filter_psi_s_2_8 = pd.DataFrame(np.zeros((M, M), dtype=float))
```

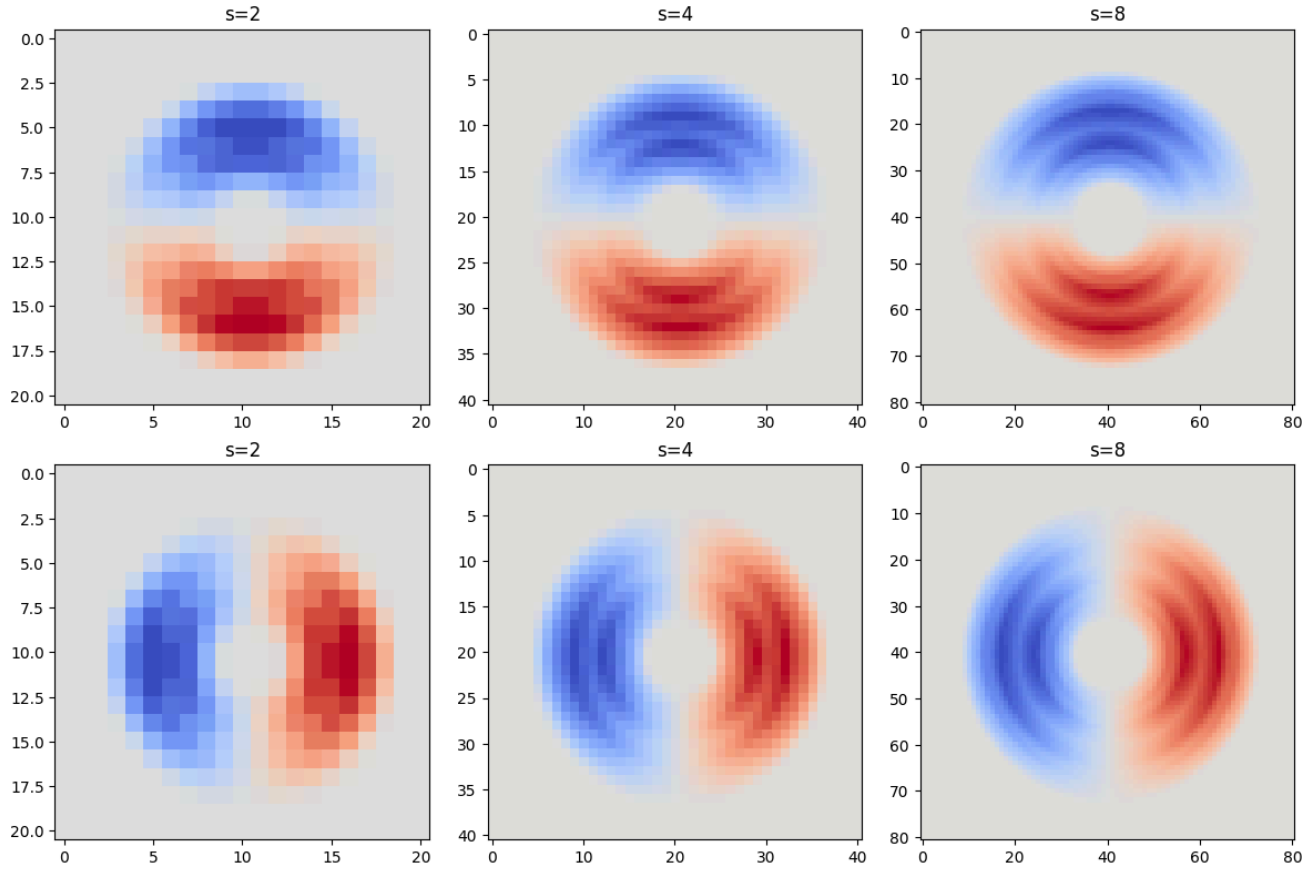
```
s = 2
k_max = np.int64(x_max * s) + 1
filter_psi_s_1_2 = pd.DataFrame(0., index = range(-k_max, k_max + 1, 1), columns = range(-k_max, k_max + 1, 1))
filter_psi_s_2_2 = pd.DataFrame(0., index = range(-k_max, k_max + 1, 1), columns = range(-k_max, k_max + 1, 1))
for k in range(-k_max, k_max + 1):
    for l in range(-k_max, k_max + 1):
        acc1 = 0.0
        acc2 = 0.0
        for i1 in range(r):
            for i2 in range(r):
                u = k - 1 + (2 * i1 + 1) / (2 * r)
                v = l - 1 + (2 * i2 + 1) / (2 * r)
                rad = math.hypot(u, v)
                theta = math.atan2(v, u)
                val_rad = psi_interp(rad / s, Lambda, X)
                val1 = (1.0 / (s * s)) * val_rad * math.cos(theta)
                val2 = (1.0 / (s * s)) * val_rad * math.sin(theta)
                acc1 += val1
                acc2 += val2
        filter_psi_s_1_2.at[k, l] = acc1 / (r * r)
        filter_psi_s_2_2.at[k, l] = acc2 / (r * r)
```

```
s = 4
k_max = np.int64(x_max * s) + 1
filter_psi_s_1_4 = pd.DataFrame(0., index = range(-k_max, k_max + 1, 1), columns = range(-k_max, k_max + 1, 1))
filter_psi_s_2_4 = pd.DataFrame(0., index = range(-k_max, k_max + 1, 1), columns = range(-k_max, k_max + 1, 1))
for k in range(-k_max, k_max + 1):
    for l in range(-k_max, k_max + 1):
        acc1 = 0.0
        acc2 = 0.0
        for i1 in range(r):
            for i2 in range(r):
                u = k - 1 + (2 * i1 + 1) / (2 * r)
                v = l - 1 + (2 * i2 + 1) / (2 * r)
                rad = math.hypot(u, v)
                theta = math.atan2(v, u)
                val_rad = psi_interp(rad / s, Lambda, X)
                val1 = (1.0 / (s * s)) * val_rad * math.cos(theta)
                val2 = (1.0 / (s * s)) * val_rad * math.sin(theta)
                acc1 += val1
                acc2 += val2
```

```
        filter_psi_s_1_4.at[k, l] = acc1 / (r * r)
        filter_psi_s_2_4.at[k, l] = acc2 / (r * r)
```

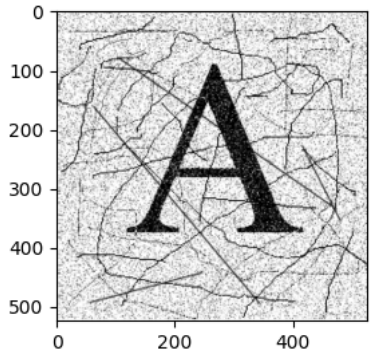
```
s = 8
k_max = np.int64(x_max * s) + 1
filter_psi_s_1_8 = pd.DataFrame(0., index = range(-k_max, k_max + 1, 1), columns = range(-k_max, k_max + 1, 1))
filter_psi_s_2_8 = pd.DataFrame(0., index = range(-k_max, k_max + 1, 1), columns = range(-k_max, k_max + 1, 1))
for k in range(-k_max, k_max + 1):
    for l in range(-k_max, k_max + 1):
        acc1 = 0.0
        acc2 = 0.0
        for i1 in range(r):
            for i2 in range(r):
                u = k - 1 + (2 * i1 + 1) / (2 * r)
                v = l - 1 + (2 * i2 + 1) / (2 * r)
                rad = math.hypot(u, v)
                theta = math.atan2(v, u)
                val_rad = psi_interp(rad / s, Lambda, X)
                val1 = (1.0 / (s * s)) * val_rad * math.cos(theta)
                val2 = (1.0 / (s * s)) * val_rad * math.sin(theta)
                acc1 += val1
                acc2 += val2
        filter_psi_s_1_8.at[k, l] = acc1 / (r * r)
        filter_psi_s_2_8.at[k, l] = acc2 / (r * r)
```

```
fig, axs = plt.subplots(2, 3, figsize=(12, 8))

im1 = axs[0,0].imshow(filter_psi_s_1_2, cmap='coolwarm')
axs[0,0].set_title('s=2')
im2 = axs[0,1].imshow(filter_psi_s_1_4, cmap='coolwarm')
axs[0,1].set_title('s=4')
im3 = axs[0,2].imshow(filter_psi_s_1_8, cmap='coolwarm')
axs[0,2].set_title('s=8')

im4 = axs[1,0].imshow(filter_psi_s_2_2, cmap='coolwarm')
axs[1,0].set_title('s=2')
im5 = axs[1,1].imshow(filter_psi_s_2_4, cmap='coolwarm')
axs[1,1].set_title('s=4')
im6 = axs[1,2].imshow(filter_psi_s_2_8, cmap='coolwarm')
axs[1,2].set_title('s=8')

plt.tight_layout()
plt.show()
```




```
f = np.array(Image.open('DetectEdge/A.PNG').convert('L'))
plt.figure(figsize=(3, 3))
plt.imshow(Image.fromarray(f), cmap='gray')
```

```
<matplotlib.image.AxesImage at 0x7ecf3ff37c80>
```



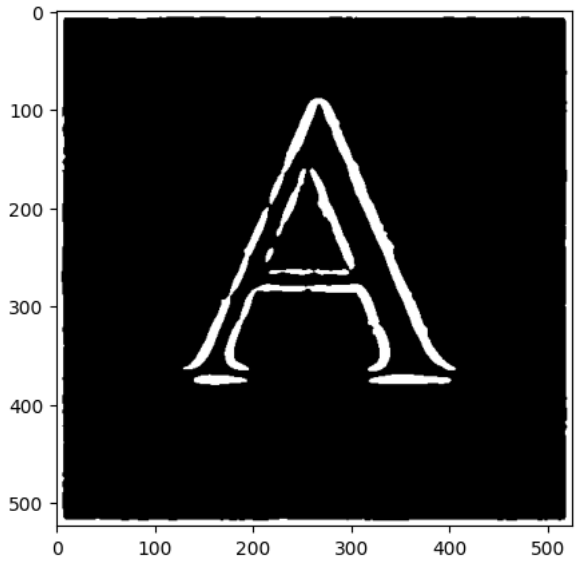
```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

```
Ws_1_2f = signal.convolve2d(f, filter_psi_s_1_2.values, mode='same', boundary='fill', fillvalue=0)
Ws_2_2f = signal.convolve2d(f, filter_psi_s_2_2.values, mode='same', boundary='fill', fillvalue=0)
Ws_1_4f = signal.convolve2d(f, filter_psi_s_1_4.values, mode='same', boundary='fill', fillvalue=0)
Ws_2_4f = signal.convolve2d(f, filter_psi_s_2_4.values, mode='same', boundary='fill', fillvalue=0)
Ws_1_8f = signal.convolve2d(f, filter_psi_s_1_8.values, mode='same', boundary='fill', fillvalue=0)
Ws_2_8f = signal.convolve2d(f, filter_psi_s_2_8.values, mode='same', boundary='fill', fillvalue=0)

Ms2 = np.sqrt((Ws_1_2f ** 2) + (Ws_2_2f ** 2))
Ms4 = np.sqrt((Ws_1_4f ** 2) + (Ws_2_4f ** 2))
Ms8 = np.sqrt((Ws_1_8f ** 2) + (Ws_2_8f ** 2))
```

```
M2_max = Ms2.max()
M4_max = Ms4.max()
M8_max = Ms8.max()

Ms2_norm = Ms2 / M2_max
Ms4_norm = Ms4 / M4_max
Ms8_norm = Ms8 / M8_max

Edge_map = (Ms2_norm + Ms4_norm + Ms8_norm) / 3
```

```
threshold = 0.55
Edges = (Edge_map > threshold).astype(np.uint8) * 255

plt.figure(figsize=(5, 5))
plt.imshow(Edges, cmap='gray')
plt.show()
```




```
# Код для задания 3 ...
```